Assignment 8

Week 9

6733172621 Patthadon Phengpinij
Collaborators. Claude (for LaTeX styling and grammar checking)

Problem 1. No-Cloning Theorem

We may also prove the No-Cloning Theorem using another method (i.e. by contradiction). We know

$$L\left(|a\rangle_A|0\rangle_B\right) = |a\rangle_A|a\rangle_B, L\left(|b\rangle_A|0\rangle_B\right) = |b\rangle_A|b\rangle_B$$

and also

$$L\left(\frac{1}{\sqrt{2}}\left(|a\rangle_A + |b\rangle_A\right)|0\rangle_B\right) = \frac{1}{\sqrt{2}}\left(|a\rangle_A + |b\rangle_A\right)\frac{1}{\sqrt{2}}\left(|a\rangle_B + |b\rangle_B\right)$$

by definition, where $|a\rangle$ and $|b\rangle$ are arbitrary normalized states. Expand the last equation and evaluate $L\left(|a\rangle_A|0\rangle_B + |b\rangle_A|0\rangle_B\right)$ to show that there is contradiction (that $|a\rangle$ and $|b\rangle$ cannot be generalized states) and thus cloning of a general state is impossible.

Solution

Cloning by definition. If L clones every state, it must produce the right-hand side of the last equation. Expanding it,

$$\begin{aligned} L\left(\tfrac{1}{\sqrt{2}}\left(|a\rangle_A + |b\rangle_A\right)|0\rangle_B\right) &= \tfrac{1}{\sqrt{2}}\left(|a\rangle_A + |b\rangle_A\right)\tfrac{1}{\sqrt{2}}\left(|a\rangle_B + |b\rangle_B\right) \\ &= \tfrac{1}{2}\left(|a\rangle_A|a\rangle_B + |a\rangle_A|b\rangle_B + |b\rangle_A|a\rangle_B + |b\rangle_A|b\rangle_B\right). \end{aligned} \tag{1}$$

Cloning by linearity. L is an operator (a matrix), so it is linear. The input can also be written as $\frac{1}{\sqrt{2}}\left(|a\rangle_A|0\rangle_B + |b\rangle_A|0\rangle_B\right)$, and distributing L over the sum uses only the first two equations:

$$\begin{aligned} L\left(\tfrac{1}{\sqrt{2}}\left(|a\rangle_A|0\rangle_B + |b\rangle_A|0\rangle_B\right)\right) &= \tfrac{1}{\sqrt{2}}\left(L\left(|a\rangle_A|0\rangle_B\right) + L\left(|b\rangle_A|0\rangle_B\right)\right) \\ &= \tfrac{1}{\sqrt{2}}\left(|a\rangle_A|a\rangle_B + |b\rangle_A|b\rangle_B\right). \end{aligned} \tag{2}$$

Both lines describe L acting on the *same* input state, so (1) and (2) must be equal.

The contradiction. The prefactor $\frac{1}{\sqrt{2}}$ normalizes $|a\rangle + |b\rangle$ when $|a\rangle$ and $|b\rangle$ are orthonormal, e.g. $|a\rangle = |0\rangle$, $|b\rangle = |1\rangle$. Then $|aa\rangle, |ab\rangle, |ba\rangle, |bb\rangle$ are the linearly independent basis states $|00\rangle, |01\rangle, |10\rangle, |11\rangle$, so (1) and (2) are equal only if every coefficient matches:

term	(1) by definition	(2) by linearity
$\|aa\rangle$	$\frac{1}{2}$	$\frac{1}{\sqrt{2}}$
$\|ab\rangle$	$\frac{1}{2}$	0
$\|ba\rangle$	$\frac{1}{2}$	0
$\|bb\rangle$	$\frac{1}{2}$	$\frac{1}{\sqrt{2}}$

No coefficient matches. The cross terms $|ab\rangle + |ba\rangle$ are the clearest problem: a true clone needs them, but linearity can never produce them. As vectors, (1) is $\frac{1}{2}(1,1,1,1)^\top = |+\rangle|+\rangle$, a product state, while (2) is $\frac{1}{\sqrt{2}}(1,0,0,1)^\top = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$, an entangled state. So an L that clones $|a\rangle$ and $|b\rangle$ *cannot* also clone their superposition. That contradicts the assumption that L clones arbitrary states $|a\rangle$, $|b\rangle$, and every superposition of them. Hence no operator clones a general state. ■

> **Remark.** The argument does not need $|a\rangle \perp |b\rangle$. A cloner must be unitary, so it preserves inner products:
>
> $$\langle a|b\rangle = (\langle a|\langle 0|)(|b\rangle|0\rangle) = (\langle a|\langle 0|)L^\dagger L(|b\rangle|0\rangle) = (\langle a|\langle a|)(|b\rangle|b\rangle) = \langle a|b\rangle^2,$$
>
> so $\langle a|b\rangle \in \{0,1\}$. One cloner can only copy states that are identical or orthogonal, i.e. the states of *one* basis (see **Problem 3**).

The notebook (**Appendix 1**) uses CNOT as a concrete L. It copies $|0\rangle$ and $|1\rangle$ perfectly, but $\text{CNOT}|+\rangle|0\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) \neq |+\rangle|+\rangle$, exactly as (2) predicts.

Problem 2. Quantum Teleportation of a Superposition State

a) Figure 1 shows the simplified version of quantum teleportation. The state to be teleported is $|+\rangle$. Explain why this is the state to be teleported?

b) Construct the matrix (4×4) from the left to Alice's first measurement.

c) Explain the simulation results and why quantum teleportation is successful.

d) Note that the MSB contains not just 0 but also 1 in the final output. Why the two Hadamard gates do not cancel each other?
Hints: Look at the matrix constructed.

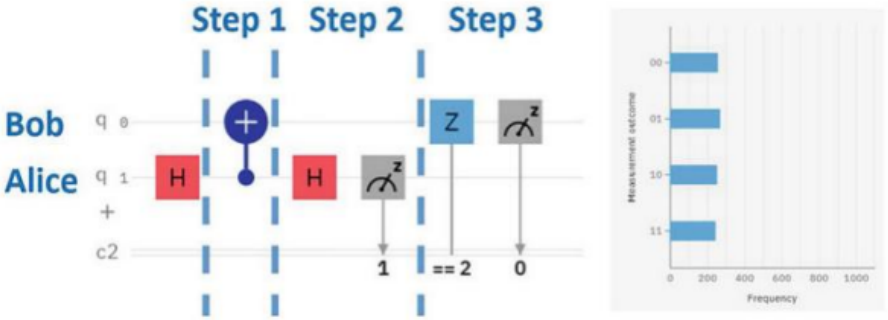


Figure 1 Quantum teleportation circuit corresponds to Pic. 20 in Chapter 18 with $|\psi\rangle = |0\rangle$

Solution

Throughout, Alice's qubit q_1 is the MSB and Bob's qubit q_0 is the LSB, so states read $|q_1 q_0\rangle = |\text{Alice}, \text{Bob}\rangle$ and the bit string is $c_1 c_0$.

(a) Why $|+\rangle$ is teleported. Every qubit starts in $|0\rangle$. The H gate on Alice's qubit, *before* the Step 1 line, is not part of the protocol. It *prepares* the state to be sent:

$$|\psi\rangle_A = H|0\rangle = \tfrac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = |+\rangle\,, \qquad \text{i.e. } \alpha = \beta = \tfrac{1}{\sqrt{2}} \text{ in Pic. 20.}$$

Without that H, Alice would send $|0\rangle$, a basis state that could simply be measured and sent classically (**Problem 6.3**). The H turns the input into a genuine superposition, so the circuit tests real teleportation.

(b) The matrix up to Alice's measurement. The gates are H on Alice, then CNOT with Alice as control (Step 1), then H on Alice (Step 2). The first gate goes on the right:

$$M = (H \otimes I)\ \text{CNOT}\ (H \otimes I)\,, \quad H \otimes I = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \end{pmatrix}, \quad \text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

Multiplying by CNOT on the left exchanges rows 3 and 4; then $H \otimes I$ adds and subtracts rows $(1, 3)$ and $(2, 4)$:

$$\text{CNOT}\,(H \otimes I) = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & -1 \\ 1 & 0 & -1 & 0 \end{pmatrix}, \qquad M = \frac{1}{2}\begin{pmatrix} 1 & 1 & 1 & -1 \\ 1 & 1 & -1 & 1 \\ 1 & -1 & 1 & 1 \\ -1 & 1 & 1 & 1 \end{pmatrix}.$$

Acting on the initial state $|00\rangle$ picks out the first column:

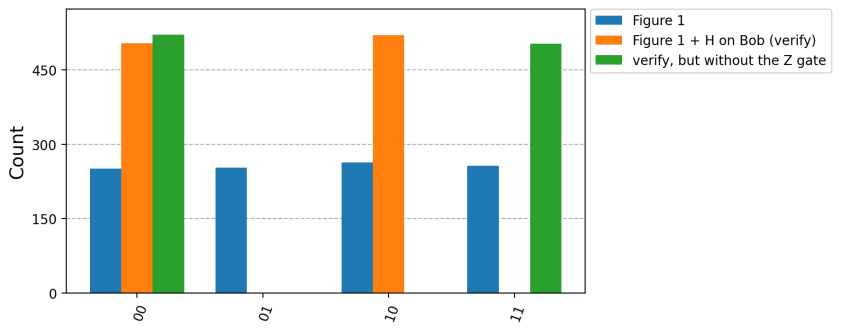
$$M|00\rangle = \tfrac{1}{2}(|00\rangle + |01\rangle + |10\rangle - |11\rangle) = \tfrac{1}{\sqrt{2}}\left(|0\rangle_A |+\rangle_B + |1\rangle_A |-\rangle_B\right).$$

This is the Step 2 equation of Pic. 20 with $\alpha = \beta = \frac{1}{\sqrt{2}}$: $|\pm\rangle_A$ has become $|0\rangle_A$ / $|1\rangle_A$ because of the second H.

(c) Simulation results. Figure 1 was built in Qiskit and run on the `AerSimulator` with 1024 shots. The composer's conditional "== 2" becomes an `if_test` on the whole register:

```
qc.h(1)                      # prepare |+> on Alice
qc.cx(1, 0)                  # Step 1
qc.h(1); qc.measure(1, 1)    # Step 2: measure Alice in the |+>/|-> basis -> c1
with qc.if_test((c, 2)):     # Step 3: Z on Bob if c == (10)_2
    qc.z(0)
qc.measure(0, 0)             # Bob -> c0
```

counts (1024 shots)	00	01	10	11
Figure 1	251	253	263	257
Figure 1, H on Bob before measuring (check)	504	0	520	0
check, but *without* the conditional Z	521	0	0	503



The four outcomes appear about equally often (≈ 256 each), as in Figure 1. From (b):

- Alice reads $c_1 = 0$ or 1 with probability $\frac{1}{2}$ each. For $c_1 = 0$, Bob already holds $|+\rangle$. For $c_1 = 1$, he holds $|-\rangle$, and the conditional Z ($c = 10_2 = 2$) turns it into $Z|-\rangle = |+\rangle$.
- Bob therefore holds $|+\rangle$ *whatever* Alice measured. Measuring $|+\rangle$ in the $|0\rangle / |1\rangle$ basis gives $c_0 = 0$ or 1 with probability $|\langle 0|+\rangle|^2 = \frac{1}{2}$.

The two bits are independent, so each of the four strings has probability $\frac{1}{4}$. Teleportation succeeds because Bob's final state is $|+\rangle$, the state Alice started with, for both of Alice's outcomes. The only extra help needed is one classical bit.

> **Remark.** A 50/50 histogram alone cannot prove that Bob holds $|+\rangle$: $|-\rangle$, or a random mix of $|0\rangle$ and $|1\rangle$, would look the same. The check in the table adds H on Bob before his measurement. Since $H|+\rangle = |0\rangle$, Bob reads $c_0 = 0$ in *every* shot, which confirms $|+\rangle$. Removing the conditional Z makes Bob's bit copy Alice's (00/11): without the classical message, Bob holds $|-\rangle$ half of the time.

(d) Why the two H gates do not cancel. If they cancelled, the circuit up to the measurement would just be the CNOT (or I). Then the MSB, which starts in $|0\rangle$, would stay $|0\rangle$, and Alice would always read 0. The matrix of (b) shows otherwise. $M \neq$ CNOT, and the first column of M has amplitude $\frac{1}{2}$ on both $|10\rangle$ and $|11\rangle$, so $\mathrm{P}(\mathrm{MSB} = 1) = \frac{1}{4} + \frac{1}{4} = \frac{1}{2}$. The reason is that the CNOT sits between the two H's and does not commute with $H \otimes I$. Writing CNOT $= |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes X$,

$$M = H|0\rangle\langle 0| H \otimes I + H|1\rangle\langle 1| H \otimes X = |+\rangle\langle +| \otimes I + |-\rangle\langle -| \otimes X,$$

which is a CNOT controlled in the $|+\rangle / |-\rangle$ basis, not the identity. Only with nothing in between would $(H \otimes I)(H \otimes I) = H^2 \otimes I = I$. Physically, the CNOT entangles Alice's qubit with Bob's, $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$. After that, Alice's qubit on its own is no longer $|+\rangle$, so the second H has no $|+\rangle$ to turn back into $|0\rangle$.

Problem 3. No-Cloning Theorem and Heisenberg Uncertainty Principle

If the state is a basis state, it is possible to clone. For any arbitrary state, it must be a basis state of another basis. Can we clone it? How is it related to the Heisenberg Uncertainty Principle? Please discuss.

Solution

Every state is a basis state of some basis. For $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, the state $|\psi^\perp\rangle = \beta^*|0\rangle - \alpha^*|1\rangle$ is orthogonal to it, so $\{|\psi\rangle, |\psi^\perp\rangle\}$ is an orthonormal basis. For example, $|+\rangle$ is a superposition in the $|0\rangle / |1\rangle$ basis but a basis state of $\{|+\rangle, |-\rangle\}$.

Can we clone it? Only if we already know that basis. If we know $|\psi\rangle$ belongs to $\{|\psi\rangle, |\psi^\perp\rangle\}$, we can measure in that basis without disturbing it, then prepare a copy. Equivalently, we can build the cloner for that basis. With $U|0\rangle = |\psi\rangle$ and $U|1\rangle = |\psi^\perp\rangle$, the gate $L_U = (U \otimes U)\ \text{CNOT}\ \left(U^\dagger \otimes U^\dagger\right)$ (blank state $U|0\rangle$) clones $|\psi\rangle$ and $|\psi^\perp\rangle$. By the remark in **Problem 1**, though, one cloner only works for mutually orthogonal states. L_U fails on every state outside its own basis. The notebook (**Appendix 1**) shows this: CNOT clones $|0\rangle, |1\rangle$ but not $|\pm\rangle$, while the $|\pm\rangle$-cloner $(H \otimes H)\,\text{CNOT}\,(H \otimes H)$ clones $|\pm\rangle$ but not $|0\rangle, |1\rangle$. For an *unknown* state, the basis it belongs to is exactly the unknown information (α, β), so there is no way to choose the right cloner. Saying "it is a basis state of some basis" does not help, and no-cloning still holds.

Relation to the Heisenberg Uncertainty Principle. The $|0\rangle / |1\rangle$ and $|+\rangle / |-\rangle$ bases are the eigenbases of σ_z and σ_x. These observables do not commute ($[\sigma_z, \sigma_x] = 2i\sigma_y$), so no state has a sharp value of both. A state that is certain in one basis is completely random in the other: $|\langle +|0\rangle|^2 = |\langle -|0\rangle|^2 = \frac{1}{2}$. A measurement in one basis also disturbs the result in the other. This is why "read, then copy" only works for basis states: reading a superposition collapses it.

- **Cloning would break the uncertainty principle.** With a perfect cloner, we could make many copies of one unknown $|\psi\rangle$, measure σ_z on some, σ_x on others, and σ_y on the rest. That would give α and β to any precision, i.e. sharp knowledge of complementary observables from a single system, without the disturbance the uncertainty principle requires.
- **The uncertainty principle makes cloning impossible.** The cloner for one basis is the measurement in that basis written as a gate. On a state of the complementary basis it must disturb the state, as $\text{CNOT}|+\rangle|0\rangle$ did in **Problem 1**, producing entanglement instead of a copy.

The two results describe the same fact: information encoded in complementary bases cannot be read, or copied, at the same time. This is also what makes quantum key distribution secure, because an eavesdropper who does not know the basis cannot copy the qubits without disturbing them.

Problem 4. Entanglement

Derive the following equation using matrix.

$$\frac{1}{\sqrt{2}}(\alpha|000\rangle + \beta|110\rangle + \alpha|011\rangle + \beta|101\rangle) = \frac{1}{\sqrt{2}}(\alpha|000\rangle + \beta|101\rangle + \alpha|011\rangle + \beta|110\rangle)$$

Solution

Setting. The equation is the 3-qubit state at the end of Step 1a of the full teleportation circuit (Pic. 21/22). The qubits are ordered $|A\,BB\,B\rangle = |q_2 q_1 q_0\rangle$, with Alice's A as the MSB, and the basis is $|000\rangle, |001\rangle, \ldots, |111\rangle$.

Initial state and Bob's Bell pair. Bob turns $|0\rangle_{BB}|0\rangle_B$ into $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$ with H on BB followed by a CNOT from BB to B. As a matrix, $\text{CNOT}\,(H \otimes I)|00\rangle = \frac{1}{\sqrt{2}}(1,0,0,1)^\top$ (first column of $\text{CNOT}\,(H \otimes I)$ in **Problem 2**). Including Alice's qubit,

$$|\Psi\rangle = |\psi\rangle_A \otimes \tfrac{1}{\sqrt{2}}(|00\rangle + |11\rangle) = \begin{pmatrix}\alpha\\ \beta\end{pmatrix} \otimes \frac{1}{\sqrt{2}}\begin{pmatrix}1\\0\\0\\1\end{pmatrix} = \frac{1}{\sqrt{2}}(\alpha,0,0,\alpha,\beta,0,0,\beta)^\top$$

$$= \tfrac{1}{\sqrt{2}}(\alpha|000\rangle + \alpha|011\rangle + \beta|100\rangle + \beta|111\rangle).$$

Alice's CNOT. A is the control and BB is the target: $U = |0\rangle\langle 0| \otimes I \otimes I + |1\rangle\langle 1| \otimes X \otimes I$.

The upper-left 4×4 block ($A = 0$) is I. The lower-right block ($A = 1$) is $X \otimes I$, which exchanges $|1\,0\,y\rangle \leftrightarrow |1\,1\,y\rangle$:

$$U|\Psi\rangle = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix} \frac{1}{\sqrt{2}} \begin{pmatrix} \alpha \\ 0 \\ 0 \\ \alpha \\ \beta \\ 0 \\ 0 \\ \beta \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} \alpha \\ 0 \\ 0 \\ \alpha \\ 0 \\ \beta \\ \beta \\ 0 \end{pmatrix} \begin{matrix} |000\rangle \\ |001\rangle \\ |010\rangle \\ |011\rangle \\ |100\rangle \\ |101\rangle \\ |110\rangle \\ |111\rangle \end{matrix}$$

Rows 5–8 are permuted: the β of $|100\rangle$ moves to $|110\rangle$, and the β of $|111\rangle$ moves to $|101\rangle$.
Reading off the result. The non-zero entries are at $|000\rangle$ and $|011\rangle$ (α) and at $|101\rangle$ and $|110\rangle$ (β), so

$$U|\Psi\rangle = \tfrac{1}{\sqrt{2}}\left(\alpha|000\rangle + \beta|110\rangle + \alpha|011\rangle + \beta|101\rangle\right) = \tfrac{1}{\sqrt{2}}\left(\alpha|000\rangle + \beta|101\rangle + \alpha|011\rangle + \beta|110\rangle\right).$$

Both sides of the equation are this same vector. The left side lists each term where it came from ($|100\rangle \to |110\rangle$, $|111\rangle \to |101\rangle$); the right side only reorders the two β terms. ■ The state is entangled: Alice's qubit and Bob's two qubits can no longer be factored apart, which Step 1b uses. The notebook (**Appendix 1**) repeats the whole calculation symbolically with 8×8 SymPy matrices.

Problem 5. Classical Register and Conditional Gate

a) Reproduce the result in Figs. 2 and 3.

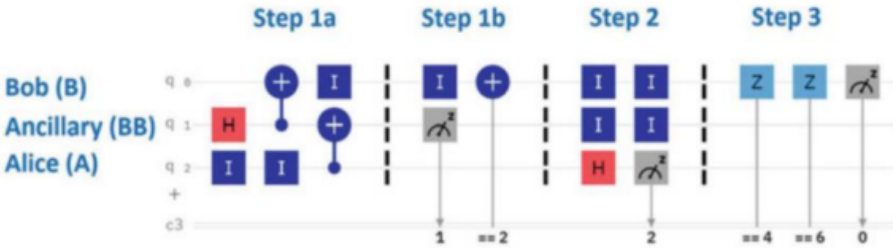


Figure 2 Implementation of the full quantum teleportation algorithm

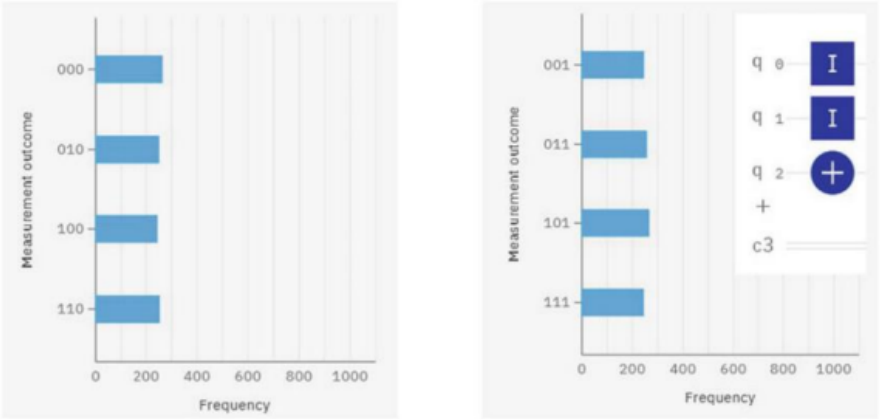


Figure 3 Simulation results of the circuit in Fig. 1 with $|\psi\rangle = |0\rangle$ (left) and $|\psi\rangle = |1\rangle$ (right). The circuit to create $|\psi\rangle = |1\rangle$ is shown as the inset

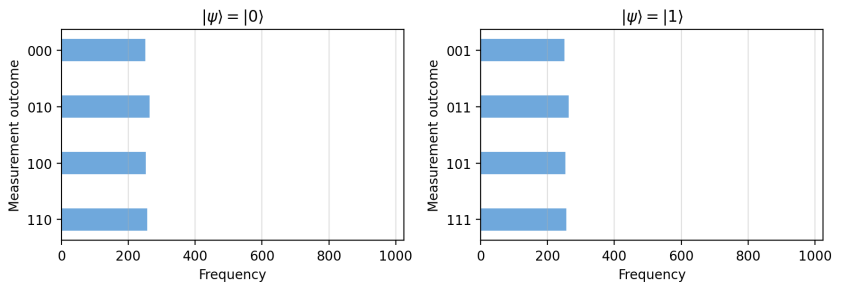
b) Store the Step 1b measurement result in the 0th classical qubit and store Step 3 measurement result in the 1st classical qubit. How should we modify the conditional CNOT and Z-gates in Step 1b and Step 3 so it will produce the same result?
Hints: The classical bit we want to inspect now is the 1st qubit.

Solution

(a) Reproducing Figures 2 and 3. Figure 2 was rebuilt in Qiskit gate by gate (q_2 = Alice, q_1 = ancillary BB, q_0 = Bob, 3-bit register c) and run on the `AerSimulator` with 1024 shots. The composer's conditional gates compare the *whole* register with a number. In Qiskit 2.x this is written as `with qc.if_test((c, value)):`

```
qc.h(1); qc.cx(1, 0); qc.cx(2, 1)    # Step 1a  (+ qc.x(2) first for |psi> = |1>)
qc.measure(1, 1)                     # Step 1b: BB -> c1
with qc.if_test((c, 2)): qc.x(0)     #          X on Bob if c == (010)_2
qc.h(2); qc.measure(2, 2)            # Step 2:  Alice in the |+>/|-> basis -> c2
with qc.if_test((c, 4)): qc.z(0)     # Step 3:  Z on Bob if c == (100)_2
with qc.if_test((c, 6)): qc.z(0)     #              or c == (110)_2
qc.measure(0, 0)                     #          Bob -> c0
```

counts (1024 shots)	000	001	010	011	100	101	110	111
$\|\psi\rangle = \|0\rangle$	251	0	263	0	253	0	257	0
$\|\psi\rangle = \|1\rangle$	0	251	0	263	0	253	0	257



This is Figure 3. For $|\psi\rangle = |0\rangle$ only `000`, `010`, `100`, `110` appear; for $|\psi\rangle = |1\rangle$ only `001`, `011`, `101`, `111`. Each appears about 256 times. Bob's bit c_0 always equals the teleported state. The bits c_1 (Step 1b) and c_2 (Step 2) are fair coin flips: they carry no information about $|\psi\rangle$ (**Problem 6.1**). The conditions work because of the order in which the bits are written:

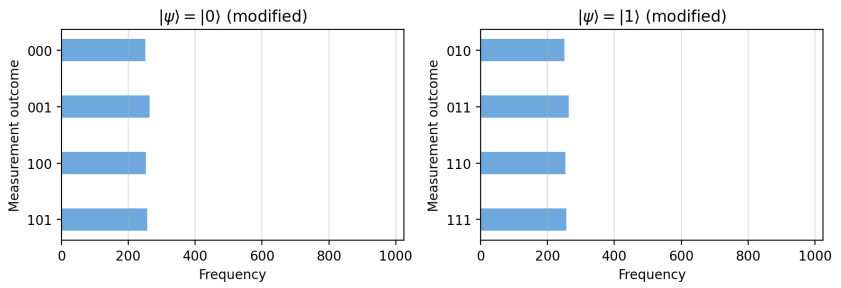
- At Step 1b only c_1 has been written, so $c = (0\,c_1\,0)_2$, and $c = 2$ exactly when BB was measured as 1.
- At Step 3, $c = (c_2\,c_1\,0)_2$. Bob needs Z whenever $c_2 = 1$, whatever c_1 is, which takes two tests: $c = (100)_2 = 4$ and $c = (110)_2 = 6$. They are never both true, so Z is applied at most once.

(b) Moving the bits. Now Step 1b writes c_0, Step 3 (Bob) writes c_1, and Step 2 still writes c_2. Each condition must be rewritten for the register value present *at that moment*:

gate	register c then	old	new
Step 1b: X on Bob	$(0\,0\,c_0)_2$	2	$(001)_2 = 1$
Step 3: Z on Bob	$(c_2\,0\,c_0)_2$	4, 6	$(100)_2 = 4$, $(101)_2 = 5$

The bit to inspect for the X gate moves from c_1 to c_0, so $2 \to 1$. For the Z gate the free bit (Step 1b's result) also moves from c_1 to c_0, so the second test changes from 6 to 5. The test $c == 4$ stays the same. Re-running with these changes (`measure(1, 0)`, `if_test((c, 1))`, `if_test((c, 4))`, `if_test((c, 5))`, `measure(0, 1)`):

counts (1024 shots)	000	001	010	011	100	101	110	111
$\|\psi\rangle = \|0\rangle$	251	263	0	0	253	257	0	0
$\|\psi\rangle = \|1\rangle$	0	0	251	263	0	0	253	257



The teleportation result is the same: Bob's bit, now c_1, always equals $|\psi\rangle$. Exchanging the labels $c_0 \leftrightarrow c_1$ gives exactly the counts of (a), which the notebook checks.

> **Remark.** The conditions must change. If only the bits are moved and the conditions stay 2 and $4, 6$, the X gate never fires (c_1 is still empty at Step 1b) and $c = 6$ can never happen. The notebook shows that Bob then gets the wrong result in half of the shots for $|\psi\rangle = |1\rangle$. The notebook also teleports three random states $R_z(\varphi)R_y(\theta)\,|0\rangle$ with both versions and undoes the preparation on Bob. Bob gets back $|0\rangle$ every time, so the circuit works for general states, not only $|0\rangle$ and $|1\rangle$.

Problem 6. Quantum Teleportation vs. Hypothetical Teleportation

Go through the quantum teleportation one more time and answer the following questions:

6.1 Why the state of the original electron is destroyed? How is it related to the no-cloning theorem?

6.2 If you want to quantum teleport myself to Moon or Mars and back to the Earth, how should you prepare for that?

6.3 If Alice's state was $|0\rangle$, do we need quantum teleportation to transfer the state to Bob? Do we violate the no-cloning theorem?

Solution

1. Why the original state is destroyed. Alice's qubit is changed twice. First, the CNOT entangles it with Bob's (or the ancillary) qubit, so on its own it is no longer in $|\psi\rangle$. Then Alice measures it in Step 2. It collapses to $|0\rangle$ or $|1\rangle$, each with probability $\frac{1}{2}$ whatever α and β are, so what remains on Alice's side says nothing about $|\psi\rangle$. The amplitudes now exist only in Bob's qubit, once he applies the correction. The notebook (**Appendix 1**) checks this: for either measurement result Alice holds $|0\rangle$ or $|1\rangle$, while Bob holds $|\psi\rangle$. This is required by the no-cloning theorem. If Alice still had $|\psi\rangle$ after Bob received it, there would be two copies of an unknown state, which **Problem 1** proves impossible. Teleportation can only *move* a state ("cut and paste"), never *copy* it ("copy and paste").

2. Teleporting yourself to the Moon or Mars and back. Following the full protocol (Figure 2), I would prepare:

- **Raw material at the destination.** Like Bob's $|0\rangle_B$, the Mars lab needs enough identical particles, in a known initial state, to hold every quantum degree of freedom of my body (more than 10^{27} atoms). Teleportation sends the *state*, not the matter.
- **Entanglement shared in advance.** Like Step 1a, one Bell pair per qubit to be teleported. One half (the ancillary BB) is carried to Earth, at normal speed, long before the trip, and stored without decoherence. Each pair is used up in one teleportation, so the return trip needs a *second* supply of pairs shared between Mars and Earth, and new raw material on Earth, arranged before I leave. Otherwise I cannot come back.
- **A classical channel.** Two classical bits per qubit (Steps 1b and 2) must reach Mars before Bob's lab can apply the X/Z corrections. Light takes about 1.3 s to the Moon and 3–22 minutes to Mars, so the trip is *not* faster than light. Until the bits arrive, the copy on Mars is in a scrambled state.
- **Near-perfect gates and measurements.** The protocol measures every particle of my

body on Earth and corrects every particle on Mars. Any gate or readout error ends up in the reconstructed person.

- **Accepting that the Earth original is destroyed.** By point 1, the particles left on Earth end in random basis states: the original "me" is gone. The person on Mars is me only if a person *is* their quantum state. The same is true on the way back, with the Mars body left behind as raw material.

3. If Alice's state is $|0\rangle$. No, quantum teleportation is not needed. $|0\rangle$ is a basis state, so Alice can measure it without changing it (or she already knows it). She sends the classical bit "0", and Bob prepares $|0\rangle$; in fact his qubit already starts in $|0\rangle_B$. This does not violate the no-cloning theorem, even though both of them end up holding $|0\rangle$. The theorem forbids copying an *arbitrary, unknown* state. As shown in **Problem 1** and on the slides, basis states *can* be cloned; CNOT does it: $\text{CNOT}\,|0\rangle\,|0\rangle = |0\rangle\,|0\rangle$. Teleportation is only needed when the state is an unknown superposition, which can neither be read nor copied.

Problem 7. Other Forms of $f(x)$

Here is what we did in f_A as below:

$$U_f\,|0\rangle\,|0\rangle = |0\rangle\,|0 \oplus f(0)\rangle = |0\rangle\,|0 \oplus 0\rangle = |0\rangle\,|0\rangle$$
$$U_f\,|0\rangle\,|1\rangle = |0\rangle\,|1 \oplus f(0)\rangle = |0\rangle\,|1 \oplus 0\rangle = |0\rangle\,|1\rangle$$
$$U_f\,|1\rangle\,|0\rangle = |1\rangle\,|0 \oplus f(1)\rangle = |1\rangle\,|0 \oplus 0\rangle = |1\rangle\,|0\rangle$$
$$U_f\,|1\rangle\,|1\rangle = |1\rangle\,|1 \oplus f(1)\rangle = |1\rangle\,|1 \oplus 0\rangle = |1\rangle\,|1\rangle$$

where,

$$f_A(0) = 0 \text{ and } f_A(1) = 0$$
$$f_B(0) = 0 \text{ and } f_B(1) = 1$$
$$f_C(0) = 1 \text{ and } f_C(1) = 0$$
$$f_D(0) = 1 \text{ and } f_D(1) = 1$$

Repeat what we did in f_A for f_B, f_C, and f_D.

Solution

The oracle keeps x (MSB) and replaces y (LSB) by $y \oplus f(x)$. XOR with 0 leaves y alone, and XOR with 1 flips it. In each matrix, column $|xy\rangle$ is the image $U_f\,|xy\rangle$ (order $00, 01, 10, 11$).

f_B: $f_B(0) = 0$, $f_B(1) = 1$.

$$U_f\,|0\rangle\,|0\rangle = |0\rangle\,|0 \oplus f(0)\rangle = |0\rangle\,|0 \oplus 0\rangle = |0\rangle\,|0\rangle \quad U_f\,|1\rangle\,|0\rangle = |1\rangle\,|0 \oplus f(1)\rangle = |1\rangle\,|0 \oplus 1\rangle = |1\rangle\,|1\rangle$$
$$U_f\,|0\rangle\,|1\rangle = |0\rangle\,|1 \oplus f(0)\rangle = |0\rangle\,|1 \oplus 0\rangle = |0\rangle\,|1\rangle \quad U_f\,|1\rangle\,|1\rangle = |1\rangle\,|1 \oplus f(1)\rangle = |1\rangle\,|1 \oplus 1\rangle = |1\rangle\,|0\rangle$$

y flips only when $x = 1$, so $U_{f_B} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} = \text{CNOT}$, with x as the control.

f_C: $f_C(0) = 1$, $f_C(1) = 0$.

$$U_f\,|0\rangle\,|0\rangle = |0\rangle\,|0 \oplus f(0)\rangle = |0\rangle\,|0 \oplus 1\rangle = |0\rangle\,|1\rangle \quad U_f\,|1\rangle\,|0\rangle = |1\rangle\,|0 \oplus f(1)\rangle = |1\rangle\,|0 \oplus 0\rangle = |1\rangle\,|0\rangle$$
$$U_f\,|0\rangle\,|1\rangle = |0\rangle\,|1 \oplus f(0)\rangle = |0\rangle\,|1 \oplus 1\rangle = |0\rangle\,|0\rangle \quad U_f\,|1\rangle\,|1\rangle = |1\rangle\,|1 \oplus f(1)\rangle = |1\rangle\,|1 \oplus 0\rangle = |1\rangle\,|1\rangle$$

y flips only when $x = 0$, so $U_{f_C} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} = (X \otimes I)\ \text{CNOT}\ (X \otimes I)$, a CNOT that fires on $x = 0$.

f_D: $f_D(0) = 1$, $f_D(1) = 1$.

$$U_f\,|0\rangle\,|0\rangle = |0\rangle\,|0 \oplus f(0)\rangle = |0\rangle\,|0 \oplus 1\rangle = |0\rangle\,|1\rangle \quad U_f\,|1\rangle\,|0\rangle = |1\rangle\,|0 \oplus f(1)\rangle = |1\rangle\,|0 \oplus 1\rangle = |1\rangle\,|1\rangle$$

$$U_f|0\rangle|1\rangle = |0\rangle|1\oplus f(0)\rangle = |0\rangle|1\oplus 1\rangle = |0\rangle|0\rangle \quad U_f|1\rangle|1\rangle = |1\rangle|1\oplus f(1)\rangle = |1\rangle|1\oplus 1\rangle = |1\rangle|0\rangle$$

y always flips, so $U_{f_D} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} = I \otimes X$, a NOT gate on the LSB.

Summary. Together with $U_{f_A} = I$, the two constant functions act on y the same way for both values of x (I or $I \otimes X$). The two balanced functions act differently depending on x: they are controlled gates. The notebook (**Appendix 1**) builds all four matrices from $|x\rangle|y \oplus f(x)\rangle$ and checks them against I, CNOT, $(X \otimes I)\,\text{CNOT}\,(X \otimes I)$, and $I \otimes X$.

Problem 8. Creation of Superposition

Repeat Eq. $(HH)(IX)|0\rangle|0\rangle = (HH)|0\rangle|1\rangle = |+\rangle|-\rangle$ using matrix.

Solution

The first factor of every tensor product acts on the MSB, the basis order is $|00\rangle, |01\rangle, |10\rangle, |11\rangle$, and the gate applied first is written on the right.

Step 1: $(I \otimes X)|0\rangle|0\rangle$**.** Each entry of I multiplies a copy of X:

$$(I \otimes X)|00\rangle = \begin{pmatrix} X & 0 \\ 0 & X \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} = |0\rangle|1\rangle.$$

Step 2: $(H \otimes H)|0\rangle|1\rangle$**.** Using $H \otimes H$ from **Assignment 7, Problem 7**,

$$(H \otimes H)|01\rangle = \frac{1}{2} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} = \frac{1}{2} \begin{pmatrix} 1 \\ -1 \\ 1 \\ -1 \end{pmatrix}.$$

Comparison with $|+\rangle|-\rangle$**.**

$$|+\rangle \otimes |-\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \otimes \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix} = \frac{1}{2} \begin{pmatrix} 1 \cdot (1, -1) \\ 1 \cdot (1, -1) \end{pmatrix} = \frac{1}{2} \begin{pmatrix} 1 \\ -1 \\ 1 \\ -1 \end{pmatrix},$$

which is the same vector. Hence

$$(HH)(IX)|0\rangle|0\rangle = (HH)|0\rangle|1\rangle = \tfrac{1}{2}(|00\rangle - |01\rangle + |10\rangle - |11\rangle) = |+\rangle|-\rangle. \qquad \blacksquare$$

The whole Step 1 as one matrix is $(H \otimes H)(I \otimes X)$: the columns of $H \otimes H$ with the pairs $(1, 2)$ and $(3, 4)$ exchanged. Its first column is again $\frac{1}{2}(1, -1, 1, -1)^\top$, as the notebook (**Appendix 1**) shows.

Problem 9. Hadamard Gate Applied to 2-Qubit Basis State

Creating the superposition in Step 1 in Fig. 4 after the NOT-gate is equivalent to applying a 2-qubit Hadamard gate to state $|01\rangle$. Show that you can get the result in the below equation by using Eq. $H^{\otimes n}|y\rangle = \frac{1}{2^{n/2}} \sum_{x=0}^{2^n-1} (-1)^{x \cdot y} |x\rangle$.

$$\begin{aligned} |+\rangle|-\rangle &= \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)\,\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \\ &= \frac{1}{2}(|00\rangle - |01\rangle + |10\rangle - |11\rangle) \end{aligned}$$

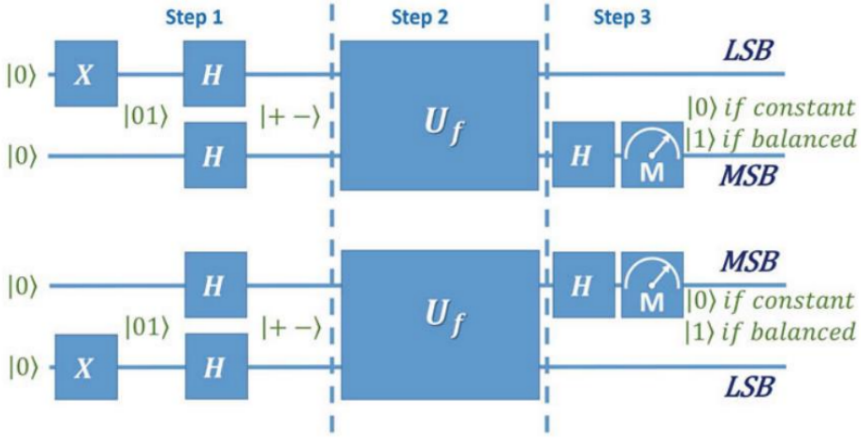


Figure 4 The Deutsch algorithm circuit. The top is the convention we have been using (IBM-Q) with MSB at the top and the bottom one is the other version often seen in other books. Both are using little-endian convention

Solution

Reading off n and y. Step 1 applies $H \otimes H = H^{\otimes 2}$ to $|01\rangle$, so $n = 2$ and $y = 01_2 = 1$, i.e. $(y_1, y_0) = (0, 1)$. The prefactor is $1/2^{n/2} = \frac{1}{2}$, and the sum runs over $x = 0, \ldots, 3$.
The signs. The bitwise product is $x \cdot y = x_1 y_1 \oplus x_0 y_0 = x_1 \cdot 0 \oplus x_0 \cdot 1 = x_0$: only the LSB of x matters.

x	$x_1 x_0$	$x \cdot y = x_1 \cdot 0 \oplus x_0 \cdot 1$	$(-1)^{x \cdot y}$	term
0	00	$0 \oplus 0 = 0$	$+1$	$+\,\|00\rangle$
1	01	$0 \oplus 1 = 1$	-1	$-\,\|01\rangle$
2	10	$0 \oplus 0 = 0$	$+1$	$+\,\|10\rangle$
3	11	$0 \oplus 1 = 1$	-1	$-\,\|11\rangle$

Result.

$$H^{\otimes 2}\,|01\rangle = \frac{1}{2}\sum_{x=0}^{3}(-1)^{x\cdot 01}\,|x\rangle = \frac{1}{2}\left(|00\rangle - |01\rangle + |10\rangle - |11\rangle\right)$$
$$= \frac{1}{\sqrt{2}}\left(|0\rangle + |1\rangle\right)\frac{1}{\sqrt{2}}\left(|0\rangle - |1\rangle\right) = |+\rangle\,|-\rangle\,,$$

which is the given equation. The minus signs sit exactly on the strings with $x_0 = 1$: the LSB, which was $|1\rangle$, becomes $|-\rangle$, while the MSB, which was $|0\rangle$, becomes $|+\rangle$ and contributes no sign. This agrees with the matrix calculation of **Problem 8**, as the notebook (**Appendix 1**) checks.

Problem 10. Non-Constant Function in the Deutsch algorithm

Repeat the following example for a balanced function. Note that in the example, $f(x)$ is a constant function that always gives "0".

$$U_f\,|0\rangle\,|0\rangle = |0\rangle\,|0 \oplus f(0)\rangle = |0\rangle\,|0 \oplus 0\rangle = |0\rangle\,|0\rangle$$
$$U_f\,|0\rangle\,|1\rangle = |0\rangle\,|1 \oplus f(0)\rangle = |0\rangle\,|1 \oplus 0\rangle = |0\rangle\,|1\rangle$$
$$U_f\,|1\rangle\,|0\rangle = |1\rangle\,|0 \oplus f(1)\rangle = |1\rangle\,|0 \oplus 0\rangle = |1\rangle\,|0\rangle$$
$$U_f\,|1\rangle\,|1\rangle = |1\rangle\,|1 \oplus f(1)\rangle = |1\rangle\,|1 \oplus 0\rangle = |1\rangle\,|1\rangle$$

Solution

The oracle. Take the balanced function f_B: $f(0) = 0$, $f(1) = 1$. Then

$$\begin{aligned}
U_f\,|0\rangle\,|0\rangle &= |0\rangle\,|0 \oplus f(0)\rangle = |0\rangle\,|0 \oplus 0\rangle = |0\rangle\,|0\rangle \\
U_f\,|0\rangle\,|1\rangle &= |0\rangle\,|1 \oplus f(0)\rangle = |0\rangle\,|1 \oplus 0\rangle = |0\rangle\,|1\rangle \\
U_f\,|1\rangle\,|0\rangle &= |1\rangle\,|0 \oplus f(1)\rangle = |1\rangle\,|0 \oplus 1\rangle = |1\rangle\,|1\rangle \\
U_f\,|1\rangle\,|1\rangle &= |1\rangle\,|1 \oplus f(1)\rangle = |1\rangle\,|1 \oplus 1\rangle = |1\rangle\,|0\rangle
\end{aligned}$$

Unlike the constant example, the oracle is no longer the identity. It exchanges $|10\rangle \leftrightarrow |11\rangle$, so it is a CNOT with x as control (**Problem 7**).

The oracle in the Deutsch algorithm. Feeding in $|+\rangle\,|-\rangle = \frac{1}{2}\left(|00\rangle - |01\rangle + |10\rangle - |11\rangle\right)$ (**Problems 8, 9**) and using the four lines above term by term,

$$\begin{aligned}
U_f\,|+\rangle\,|-\rangle &= \tfrac{1}{2}\left(U_f\,|00\rangle - U_f\,|01\rangle + U_f\,|10\rangle - U_f\,|11\rangle\right) = \tfrac{1}{2}\left(|00\rangle - |01\rangle + |11\rangle - |10\rangle\right) \\
&= \tfrac{1}{2}\left(|0\rangle - |1\rangle\right)\left(|0\rangle - |1\rangle\right) = |-\rangle\,|-\rangle\,.
\end{aligned}$$

The constant oracle left $|+\rangle\,|-\rangle$ unchanged. The balanced oracle leaves the LSB as $|-\rangle$ but flips the MSB from $|+\rangle$ to $|-\rangle$: the -1 is "kicked back" onto the input qubit. Step 3 applies H to the MSB, $H\,|-\rangle = |1\rangle$, so the measurement gives 1 with certainty, i.e. "balanced", after a single use of the oracle.

Simulation. Running the Deutsch circuit of Figure 4 (1024 shots on the `AerSimulator`) with each oracle of **Problem 7** built from gates gives

oracle	f_A (constant)	f_B (balanced)	f_C (balanced)	f_D (constant)
MSB measured `0`	1024	0	0	1024
MSB measured `1`	0	1024	1024	0

The balanced f_B (and f_C) gives `1` in every shot, while the constant functions give `0`, as predicted.

APPENDIX 1

Assignment 08 | Code Solution

assignment-08-code

October 19, 2026

1 Assignment 08

1.1 Import External Libraries

[1]:
```
import numpy as np
import sympy as sp
import matplotlib.pyplot as plt

from IPython.display import display, Math

from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_histogram
from qiskit_aer import AerSimulator
```

1.2 1. No-Cloning Theorem

[2]:
```
# --- Problem 1: cloning by linearity vs. cloning by definition ---
def latex_matrix(M):
    """LaTeX for a numeric array: rounded to 4 d.p., whole numbers shown as integers."""
    M = np.round(np.real_if_close(np.asarray(M, dtype=complex)), 4) + 0.0
    return sp.latex(sp.Matrix(M).applyfunc(lambda v: sp.Integer(int(v)) if float(v).is_integer()␣
  ↪else v))


I2 = np.eye(2)
X  = np.array([[0, 1], [1, 0]])
Z  = np.array([[1, 0], [0, -1]])
H  = np.array([[1, 1], [1, -1]]) / np.sqrt(2)
P0 = np.array([[1, 0], [0, 0]])   # |0><0|
P1 = np.array([[0, 0], [0, 1]])   # |1><1|

ket0, ket1 = np.array([1, 0]), np.array([0, 1])
plus, minus = (ket0 + ket1) / np.sqrt(2), (ket0 - ket1) / np.sqrt(2)

# Symbolic check with |a> = |0>, |b> = |1> (orthonormal, so |aa>, |ab>, |ba>, |bb> are␣
  ↪independent)
aa, ab, ba, bb = sp.symbols("aa ab ba bb")   # stand-ins for the four product kets
by_definition = sp.expand(sp.Rational(1, 2) * (aa + ab + ba + bb))
by_linearity  = (aa + bb) / sp.sqrt(2)
display(Math(r"\text{definition: } " + sp.latex(by_definition)))
display(Math(r"\text{linearity: } " + sp.latex(by_linearity)))
print("Problem 1  --  coefficient of |ab>:  definition", by_definition.coeff(ab), "  linearity",␣
  ↪by_linearity.coeff(ab))

print("---------------")
```

```
# A concrete candidate cloner: CNOT (A controls, B is the target) copies |0> and |1> ...
CNOT = np.kron(P0, I2) + np.kron(P1, X)
print("Problem 1  --  CNOT|0>|0> == |0>|0> ?", np.allclose(CNOT @ np.kron(ket0, ket0), np.
  ↪kron(ket0, ket0)))
print("Problem 1  --  CNOT|1>|0> == |1>|1> ?", np.allclose(CNOT @ np.kron(ket1, ket0), np.
  ↪kron(ket1, ket1)))
# ... but not their superposition |+>: it gives (|00> + |11>)/sqrt2, not |+>|+>
out = CNOT @ np.kron(plus, ket0)
display(Math(r"\text{CNOT}\ket{+}\ket{0} = " + latex_matrix(out.reshape(1, -1))
             + r",\quad \ket{+}\ket{+} = " + latex_matrix(np.kron(plus, plus).reshape(1, -1))))
print("Problem 1  --  CNOT|+>|0> == |+>|+> ?", np.allclose(out, np.kron(plus, plus)))

print("---------------")

# Unitarity: L preserves inner products, so <a|b> = <a|b>^2 for any two clonable states
s = sp.symbols("s")
print("Problem 1  --  solutions of <a|b> = <a|b>^2 :", sp.solve(sp.Eq(s, s**2), s))
```

definition: $\frac{aa}{2} + \frac{ab}{2} + \frac{ba}{2} + \frac{bb}{2}$

linearity: $\frac{\sqrt{2}\,(aa + bb)}{2}$

```
Problem 1  --  coefficient of |ab>:  definition 1/2   linearity 0
---------------
Problem 1  --  CNOT|0>|0> == |0>|0> ? True
Problem 1  --  CNOT|1>|0> == |1>|1> ? True
```

$\mathrm{CNOT}\,|+\rangle\,|0\rangle = \begin{bmatrix}0.7071 & 0 & 0 & 0.7071\end{bmatrix}, \quad |+\rangle\,|+\rangle = \begin{bmatrix}0.5 & 0.5 & 0.5 & 0.5\end{bmatrix}$

```
Problem 1  --  CNOT|+>|0> == |+>|+> ? False
---------------
Problem 1  --  solutions of <a|b> = <a|b>^2 : [0, 1]
```

1.3 2. Quantum Teleportation of a Superposition State

[3]:
```
# --- Problem 2: the simplified teleportation circuit of Figure 1 (q1 = Alice = MSB, q0 = Bob =␣
  ↪LSB) ---
H_M  = np.kron(H, I2)          # H on Alice (MSB is the first tensor factor)
M_2b = H_M @ CNOT @ H_M        # gates in time order H, CNOT, H  ->  product with the first gate␣
  ↪on the right
display(Math(r"(H \otimes I)\,\text{CNOT}\,(H \otimes I) = " + latex_matrix(M_2b)))
print("Problem 2  --  equals 1/2 [[1,1,1,-1],[1,1,-1,1],[1,-1,1,1],[-1,1,1,1]] ?",
      np.allclose(2 * M_2b, [[1, 1, 1, -1], [1, 1, -1, 1], [1, -1, 1, 1], [-1, 1, 1, 1]]))
print("Problem 2  --  the two H's cancel (product == CNOT) ?", np.allclose(M_2b, CNOT))

state = M_2b @ np.kron(ket0, ket0)
display(Math(r"(H \otimes I)\,\text{CNOT}\,(H \otimes I)\ket{00} = " + latex_matrix(state.
  ↪reshape(1, -1))))
print("Problem 2  --  state == (|0>|+> + |1>|->)/sqrt2 ?",
      np.allclose(state, (np.kron(ket0, plus) + np.kron(ket1, minus)) / np.sqrt(2)))
print("Problem 2  --  after Alice's H, Alice's qubit is |+> ?", np.allclose(H @ ket0, plus))
```

$$(H \otimes I)\,\mathrm{CNOT}\,(H \otimes I) = \begin{bmatrix} 0.5 & 0.5 & 0.5 & -0.5 \\ 0.5 & 0.5 & -0.5 & 0.5 \\ 0.5 & -0.5 & 0.5 & 0.5 \\ -0.5 & 0.5 & 0.5 & 0.5 \end{bmatrix}$$

```
Problem 2  --  equals 1/2 [[1,1,1,-1],[1,1,-1,1],[1,-1,1,1],[-1,1,1,1]] ? True
Problem 2  --  the two H's cancel (product == CNOT) ? False
```

$(H \otimes I)\,\mathrm{CNOT}\,(H \otimes I)\,|00\rangle = \begin{bmatrix}0.5 & 0.5 & 0.5 & -0.5\end{bmatrix}$

```
Problem 2  --  state == (|0>|+> + |1>|->)/sqrt2 ? True
Problem 2  --  after Alice's H, Alice's qubit is |+> ? True
```

[4]:
```python
# --- Problem 2: simulate Figure 1 (and a check that Bob really holds |+>) ---
SHOTS = 1024
sim = AerSimulator(seed_simulator=2110515)


def figure1(verify=False, correct=True):
    """Figure 1. verify: H on Bob before his measurement (|+> -> |0>). correct: keep the␣
  ↪conditional Z."""
    c  = ClassicalRegister(2, "c")
    qc = QuantumCircuit(QuantumRegister(2, "q"), c)
    qc.h(1)                       # prepare Alice's state |+>
    qc.barrier()
    qc.cx(1, 0)                   # Step 1: entangle Alice (control) with Bob
    qc.barrier()
    qc.h(1)                       # Step 2: measure Alice in the |+>/|-> basis
    qc.measure(1, 1)
    qc.barrier()
    if correct:
        with qc.if_test((c, 2)):  # Step 3: Bob applies Z if c == (10)_2
            qc.z(0)
    if verify:
        qc.h(0)
    qc.measure(0, 0)
    return qc

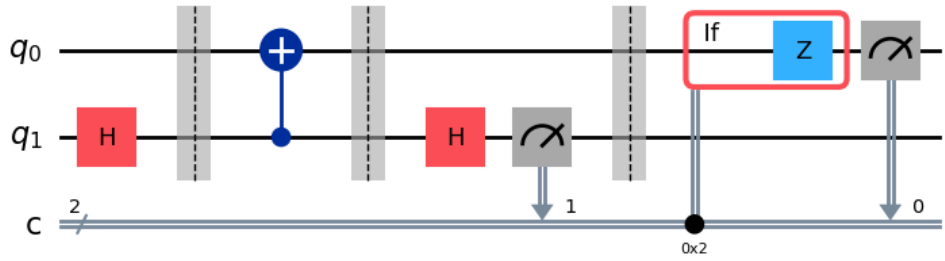

def counts(qc):
    return sim.run(qc, shots=SHOTS).result().get_counts()


def report(cnt, label, nbits):
    print(label)
    for k in range(2 ** nbits):
        key = format(k, f"0{nbits}b")
        print(f"  {key}:  {cnt.get(key, 0):4d} / {SHOTS}")

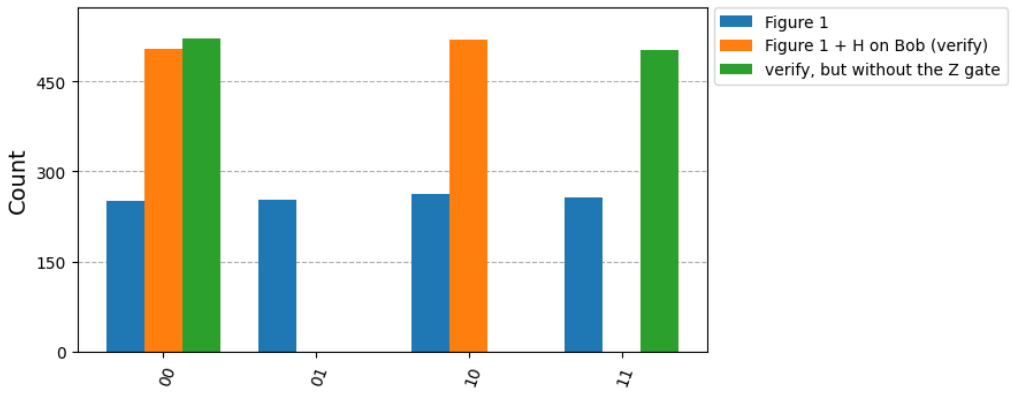

qc = figure1()
display(qc.draw("mpl"))
plt.close("all")

runs = {
    "Figure 1":                          counts(figure1()),
    "Figure 1 + H on Bob (verify)":      counts(figure1(verify=True)),
    "verify, but without the Z gate":    counts(figure1(verify=True, correct=False)),
}
for label, cnt in runs.items():
    report(cnt, label + "  (bit string c1 c0 = Alice, Bob)", 2)

fig = plot_histogram(list(runs.values()), legend=list(runs), figsize=(9, 3.6), bar_labels=False)
fig.savefig("../images/problem-2-answer.png", dpi=200, bbox_inches="tight")
display(fig)
plt.close(fig)
```




```
Figure 1  (bit string c1 c0 = Alice, Bob)
  00:   251 / 1024
  01:   253 / 1024
  10:   263 / 1024
  11:   257 / 1024
Figure 1 + H on Bob (verify)  (bit string c1 c0 = Alice, Bob)
  00:   504 / 1024
  01:     0 / 1024
  10:   520 / 1024
  11:     0 / 1024
verify, but without the Z gate  (bit string c1 c0 = Alice, Bob)
  00:   521 / 1024
  01:     0 / 1024
  10:     0 / 1024
  11:   503 / 1024
```



1.4 3. No-Cloning Theorem and Heisenberg Uncertainty Principle

[5]:
```python
# --- Problem 3: a cloner made for one basis fails in the other ---
# CNOT clones the Z basis {|0>, |1>}; the cloner for the X basis {|+>, |->} is (H(x)H) CNOT (H(x)H)
HH = np.kron(H, H)
CNOT_X = HH @ CNOT @ HH
for name, L in (("Z-basis cloner", CNOT), ("X-basis cloner", CNOT_X)):
    for kname, k in (("|0>", ket0), ("|1>", ket1), ("|+>", plus), ("|->", minus)):
        out = L @ np.kron(k, ket0 if L is CNOT else plus)   # blank state: |0> for Z, |+> for X
        print(f"Problem 3  --  {name} on {kname}:  clone succeeds? {np.allclose(out, np.kron(k, k))}")
```

```
    print("---------------")

# |<0|+>|^2 = 1/2: a state definite in Z is maximally uncertain in X
print("Problem 3  --  |<0|+>|^2 =", round(abs(ket0 @ plus) ** 2, 4))
```

```
Problem 3  --  Z-basis cloner on |0>:  clone succeeds? True
Problem 3  --  Z-basis cloner on |1>:  clone succeeds? True
Problem 3  --  Z-basis cloner on |+>:  clone succeeds? False
Problem 3  --  Z-basis cloner on |->:  clone succeeds? False
---------------
Problem 3  --  X-basis cloner on |0>:  clone succeeds? False
Problem 3  --  X-basis cloner on |1>:  clone succeeds? False
Problem 3  --  X-basis cloner on |+>:  clone succeeds? True
Problem 3  --  X-basis cloner on |->:  clone succeeds? True
---------------
Problem 3  --  |<0|+>|^2 = 0.5
```

1.5 4. Entanglement

[6]:
```
# --- Problem 4: state after Step 1a (q2 = A = MSB, q1 = BB, q0 = B), with 8x8 matrices ---
al, be = sp.symbols("alpha beta")
sI, sX, sH = sp.eye(2), sp.Matrix([[0, 1], [1, 0]]), sp.Matrix([[1, 1], [1, -1]]) / sp.sqrt(2)
sP0, sP1 = sp.Matrix([[1, 0], [0, 0]]), sp.Matrix([[0, 0], [0, 1]])
kron = lambda *ms: sp.kronecker_product(*ms)


def sket(bits):
    v = sp.zeros(2 ** len(bits), 1)
    v[int(bits, 2)] = 1
    return v


H_BB      = kron(sI, sH, sI)                              # H on the ancillary qubit BB
CNOT_BB_B = kron(sI, sP0, sI) + kron(sI, sP1, sX)         # BB controls, B is the target
CNOT_A_BB = kron(sP0, sI, sI) + kron(sP1, sX, sI)         # A controls, BB is the target

psi_in = al * sket("000") + be * sket("100")              # (alpha|0> + beta|1>)_A |0>_BB |0>_B
before = CNOT_BB_B * H_BB * psi_in                        # |psi>_A (|00> + |11>)/sqrt2
after  = CNOT_A_BB * before

display(Math(r"\text{CNOT}_{A \to BB} = " + sp.latex(CNOT_A_BB)))
display(Math(r"\text{before} = " + sp.latex(before.T)))
display(Math(r"\text{after} = " + sp.latex(after.T)))

lhs = (al * sket("000") + be * sket("110") + al * sket("011") + be * sket("101")) / sp.sqrt(2)
rhs = (al * sket("000") + be * sket("101") + al * sket("011") + be * sket("110")) / sp.sqrt(2)
print("Problem 4  --  after == LHS ?", sp.simplify(after - lhs) == sp.zeros(8, 1))
print("Problem 4  --  after == RHS ?", sp.simplify(after - rhs) == sp.zeros(8, 1))
```

$$\text{CNOT}_{A\to BB} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$\text{before} = \begin{bmatrix} \frac{\sqrt{2}\alpha}{2} & 0 & 0 & \frac{\sqrt{2}\alpha}{2} & \frac{\sqrt{2}\beta}{2} & 0 & 0 & \frac{\sqrt{2}\beta}{2} \end{bmatrix}$$

$$\text{after} = \begin{bmatrix} \frac{\sqrt{2}\alpha}{2} & 0 & 0 & \frac{\sqrt{2}\alpha}{2} & 0 & \frac{\sqrt{2}\beta}{2} & \frac{\sqrt{2}\beta}{2} & 0 \end{bmatrix}$$

```
Problem 4  --  after == LHS ? True
Problem 4  --  after == RHS ? True
```

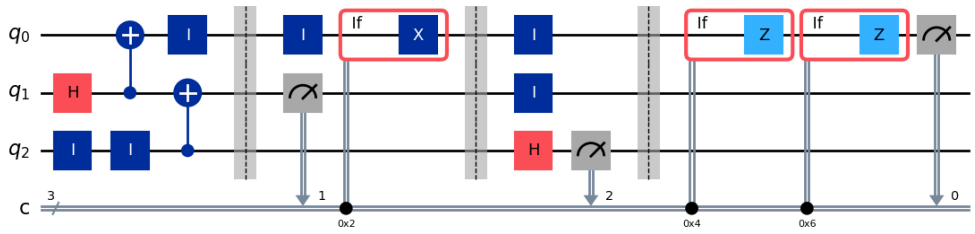
1.6 5. Classical Register and Conditional Gate

[7]:
```
# --- Problem 5: the full teleportation circuit of Figure 2 (q2 = Alice, q1 = ancillary BB, q0 =␣
  ↪Bob) ---
def figure2(psi_is_1, bit_1b=1, bit_3=0, x_when=2, z_when=(4, 6)):
    """Figure 2. bit_1b / bit_3: classical bits for the Step 1b / Step 3 measurements;
    x_when / z_when: register values that trigger Bob's conditional X / Z gates."""
    c  = ClassicalRegister(3, "c")
    qc = QuantumCircuit(QuantumRegister(3, "q"), c)
    if psi_is_1:
        qc.x(2)                          # the inset of Figure 3: |psi> = |1>
    # Step 1a
    qc.h(1)
    qc.id(2)
    qc.cx(1, 0)
    qc.id(2)
    qc.cx(2, 1)
    qc.id(0)
    qc.barrier()
    # Step 1b
    qc.id(0)
    qc.measure(1, bit_1b)
    with qc.if_test((c, x_when)):
        qc.x(0)
    qc.barrier()
    # Step 2
    qc.id([0, 1])
    qc.h(2)
    qc.measure(2, 2)
    qc.barrier()
    # Step 3
    for v in z_when:
        with qc.if_test((c, v)):
            qc.z(0)
    qc.measure(0, bit_3)
    return qc

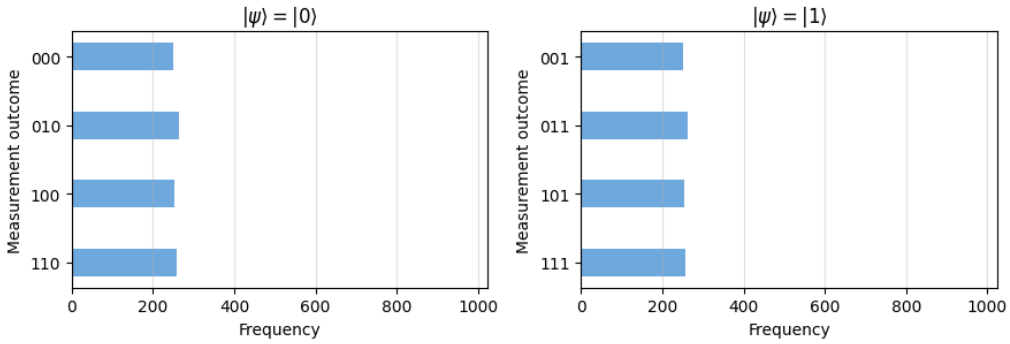

display(figure2(psi_is_1=False).draw("mpl", fold=-1))
plt.close("all")


def side_by_side(cnts, titles, filename):
    """Figure 3 style: one horizontal histogram per input state."""
    fig, axes = plt.subplots(1, len(cnts), figsize=(9, 3.2))
    for ax, cnt, title in zip(axes, cnts, titles):
        keys = sorted(cnt, reverse=True)
        ax.barh(keys, [cnt[k] for k in keys], color="#6fa8dc", height=0.4)
        ax.set_xlim(0, SHOTS)
        ax.set_xlabel("Frequency")
        ax.set_ylabel("Measurement outcome")
        ax.set_title(title)
        ax.grid(axis="x", alpha=0.4)
    fig.tight_layout()
    fig.savefig(f"../images/{filename}", dpi=200, bbox_inches="tight")
    display(fig)
    plt.close(fig)
```

```
fig2_counts = [counts(figure2(psi_is_1=p)) for p in (False, True)]
for p, cnt in zip((0, 1), fig2_counts):
    report(cnt, f"Figure 2 with |psi> = |{p}>  (bit string c2 c1 c0)", 3)
    print(f"Problem 5a  --  Bob's bit c0 is always {p} ?", all(k[-1] == str(p) for k in cnt))
side_by_side(fig2_counts, [r"$|\psi\rangle = |0\rangle$", r"$|\psi\rangle = |1\rangle$"],␣
  ↪"problem-5a-answer.png")
```




```
Figure 2 with |psi> = |0>  (bit string c2 c1 c0)
  000:   251 / 1024
  001:     0 / 1024
  010:   263 / 1024
  011:     0 / 1024
  100:   253 / 1024
  101:     0 / 1024
  110:   257 / 1024
  111:     0 / 1024
Problem 5a  --  Bob's bit c0 is always 0 ? True
Figure 2 with |psi> = |1>  (bit string c2 c1 c0)
  000:     0 / 1024
  001:   251 / 1024
  010:     0 / 1024
  011:   263 / 1024
  100:     0 / 1024
  101:   253 / 1024
  110:     0 / 1024
  111:   257 / 1024
Problem 5a  --  Bob's bit c0 is always 1 ? True
```



[8]:
```
# --- Problem 5b: Step 1b -> c0, Step 3 -> c1; conditions move to c == 1 (X) and c == 4, 5 (Z) ---
mod = dict(bit_1b=0, bit_3=1, x_when=1, z_when=(4, 5))
display(figure2(psi_is_1=False, **mod).draw("mpl", fold=-1))
```

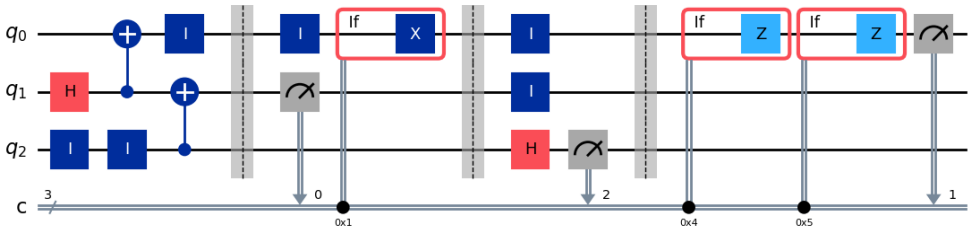
```
plt.close("all")

mod_counts = [counts(figure2(psi_is_1=p, **mod)) for p in (False, True)]
for p, cnt in zip((0, 1), mod_counts):
    report(cnt, f"Modified Figure 2 with |psi> = |{p}>  (bit string c2 c1 c0)", 3)
    print(f"Problem 5b  --  Bob's bit c1 is always {p} ?", all(k[1] == str(p) for k in cnt))
side_by_side(mod_counts, [r"$|\psi\rangle = |0\rangle$ (modified)", r"$|\psi\rangle = |1\rangle$ (modified)"],
             "problem-5b-answer.png")

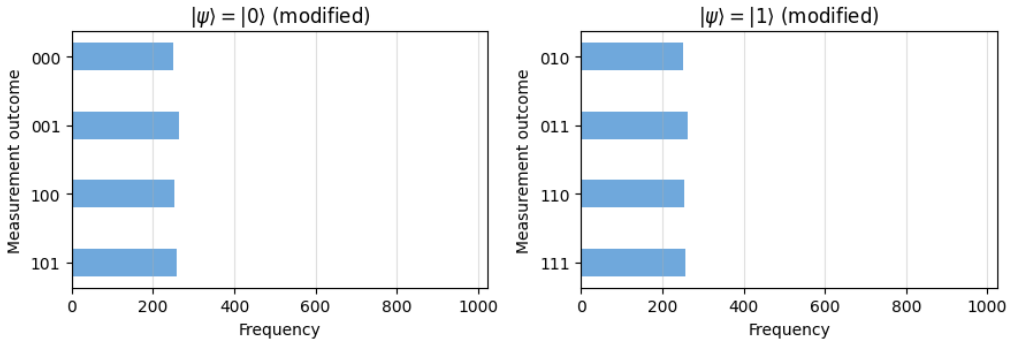
# the same outcomes once c0 and c1 are swapped back
swap01 = lambda cnt: {k[0] + k[2] + k[1]: v for k, v in cnt.items()}
print("Problem 5b  --  identical to 5a after relabelling c0 <-> c1 ?",
      all(swap01(m) == o for m, o in zip(mod_counts, fig2_counts)))

print("---------------")

# keeping the old conditions (x_when=2, z_when=(4, 6)) breaks the teleportation of |1>
broken = counts(figure2(psi_is_1=True, bit_1b=0, bit_3=1))
report(broken, "Bits moved but conditions NOT updated, |psi> = |1>", 3)
```




```
Modified Figure 2 with |psi> = |0>  (bit string c2 c1 c0)
  000:   251 / 1024
  001:   263 / 1024
  010:     0 / 1024
  011:     0 / 1024
  100:   253 / 1024
  101:   257 / 1024
  110:     0 / 1024
  111:     0 / 1024
Problem 5b  --  Bob's bit c1 is always 0 ? True
Modified Figure 2 with |psi> = |1>  (bit string c2 c1 c0)
  000:     0 / 1024
  001:     0 / 1024
  010:   251 / 1024
  011:   263 / 1024
  100:     0 / 1024
  101:     0 / 1024
  110:   253 / 1024
  111:   257 / 1024
Problem 5b  --  Bob's bit c1 is always 1 ? True
```




```
Problem 5b  --  identical to 5a after relabelling c0 <-> c1 ? True
---------------
Bits moved but conditions NOT updated, |psi> = |1>
  000:     0 / 1024
  001:   263 / 1024
  010:   251 / 1024
  011:     0 / 1024
  100:     0 / 1024
  101:   257 / 1024
  110:   253 / 1024
  111:     0 / 1024
```

[9]:
```python
# --- Problem 5 (extra): both versions teleport a general state, not just |0> and |1> ---
def teleports(theta, phi, **kw):
    """Prepare Ry(theta) Rz(phi)|0> on Alice, teleport, undo the preparation on Bob: Bob must read 0."""
    qc = figure2(psi_is_1=False, **kw)
    prep = QuantumCircuit(3)
    prep.ry(theta, 2)
    prep.rz(phi, 2)
    qc = qc.compose(prep, front=True)
    # undo the preparation on Bob just before his measurement (the last instruction)
    last = qc.data.pop()
    qc.rz(-phi, 0)
    qc.ry(-theta, 0)
    qc.append(last)
    return counts(qc)


rng = np.random.default_rng(2110515)
for theta, phi in rng.uniform(0, np.pi, size=(3, 2)):
    for name, kw in (("Figure 2", {}), ("modified", mod)):
        bob = kw.get("bit_3", 0)   # Bob's classical bit; bit strings read c2 c1 c0
        ok = all(k[2 - bob] == "0" for k in teleports(theta, phi, **kw))
        print(f"Problem 5  --  theta={theta:.3f}, phi={phi:.3f}, {name:9s}: Bob always gets back |0> ? {ok}")
```

```
Problem 5  --  theta=1.762, phi=0.164, Figure 2 : Bob always gets back |0> ?
True
Problem 5  --  theta=1.762, phi=0.164, modified : Bob always gets back |0> ?
True
Problem 5  --  theta=1.451, phi=2.587, Figure 2 : Bob always gets back |0> ?
True
Problem 5  --  theta=1.451, phi=2.587, modified : Bob always gets back |0> ?
True
Problem 5  --  theta=2.765, phi=0.654, Figure 2 : Bob always gets back |0> ?
True
```

```
Problem 5  --  theta=2.765, phi=0.654, modified : Bob always gets back |0> ?
True
```

1.7 6. Quantum Teleportation vs. Hypothetical Teleportation

[10]:
```
# --- Problem 6: after Step 2 Alice's qubit is |0> or |1> -- the amplitudes alpha, beta are gone␣
  ↪from her side ---
theta = 1.0
psi = np.array([np.cos(theta / 2), np.sin(theta / 2)])   # an "unknown" state alpha|0> + beta|1>
qc = QuantumCircuit(2)
qc.initialize(psi, 1)
qc.cx(1, 0)
qc.h(1)
sv = Statevector(qc)
for a in (0, 1):
    # project Alice (q1) onto |a>; Bob's conditional state, Z-corrected when a = 1
    bob = np.array([sv.data[2 * a + 0], sv.data[2 * a + 1]])
    bob = bob / np.linalg.norm(bob)
    if a == 1:
        bob = Z @ bob
    print(f"Problem 6  --  Alice reads {a}: Alice holds |{a}>, Bob (after correction) holds psi ?␣
  ↪{np.allclose(bob, psi)}")
```

```
Problem 6  --  Alice reads 0: Alice holds |0>, Bob (after correction) holds psi
? True
Problem 6  --  Alice reads 1: Alice holds |1>, Bob (after correction) holds psi
? True
```

1.8 7. Other Forms of f(x)

[11]:
```
# --- Problem 7: U_f |x>|y> = |x>|y xor f(x)> for the four one-bit functions ---
def basis(bits):
    v = np.zeros(2 ** len(bits))
    v[int(bits, 2)] = 1
    return v


def U_f(f):
    """Oracle matrix whose column |x y> is |x, y xor f(x)>."""
    return np.column_stack([basis(f"{x}{y ^ f(x)}") for x in (0, 1) for y in (0, 1)])


FUNCS = {"A": lambda x: 0, "B": lambda x: x, "C": lambda x: 1 - x, "D": lambda x: 1}
EXPECTED = {"A": np.eye(4), "B": CNOT, "C": np.kron(X, I2) @ CNOT @ np.kron(X, I2), "D": np.
  ↪kron(I2, X)}

for name, f in FUNCS.items():
    print(f"f_{name}:  f(0) = {f(0)}, f(1) = {f(1)}")
    for x in (0, 1):
        for y in (0, 1):
            print(f"  U_f |{x}>|{y}> = |{x}>|{y} xor {f(x)}> = |{x}>|{y ^ f(x)}>")
    display(Math(rf"U_{{f_{name}}} = " + latex_matrix(U_f(f))))
    print(f"Problem 7  --  U_f_{name} matches I / CNOT / anti-CNOT / I(x)X ?", np.
  ↪allclose(U_f(f), EXPECTED[name]))
    print("---------------")
```

```
f_A:  f(0) = 0, f(1) = 0
  U_f |0>|0> = |0>|0 xor 0> = |0>|0>
```

```
  U_f |0>|1> = |0>|1 xor 0> = |0>|1>
  U_f |1>|0> = |1>|0 xor 0> = |1>|0>
  U_f |1>|1> = |1>|1 xor 0> = |1>|1>
```

$$U_{f_A} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

```
Problem 7  --  U_f_A matches I / CNOT / anti-CNOT / I(x)X ? True
---------------
f_B:  f(0) = 0, f(1) = 1
  U_f |0>|0> = |0>|0 xor 0> = |0>|0>
  U_f |0>|1> = |0>|1 xor 0> = |0>|1>
  U_f |1>|0> = |1>|0 xor 1> = |1>|1>
  U_f |1>|1> = |1>|1 xor 1> = |1>|0>
```

$$U_{f_B} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

```
Problem 7  --  U_f_B matches I / CNOT / anti-CNOT / I(x)X ? True
---------------
f_C:  f(0) = 1, f(1) = 0
  U_f |0>|0> = |0>|0 xor 1> = |0>|1>
  U_f |0>|1> = |0>|1 xor 1> = |0>|0>
  U_f |1>|0> = |1>|0 xor 0> = |1>|0>
  U_f |1>|1> = |1>|1 xor 0> = |1>|1>
```

$$U_{f_C} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

```
Problem 7  --  U_f_C matches I / CNOT / anti-CNOT / I(x)X ? True
---------------
f_D:  f(0) = 1, f(1) = 1
  U_f |0>|0> = |0>|0 xor 1> = |0>|1>
  U_f |0>|1> = |0>|1 xor 1> = |0>|0>
  U_f |1>|0> = |1>|0 xor 1> = |1>|1>
  U_f |1>|1> = |1>|1 xor 1> = |1>|0>
```

$$U_{f_D} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

```
Problem 7  --  U_f_D matches I / CNOT / anti-CNOT / I(x)X ? True
---------------
```

1.9 8. Creation of Superposition

[12]:
```
# --- Problem 8: (H H)(I X)|0>|0> = (H H)|0>|1> = |+>|-> ---
IX, HH = np.kron(I2, X), np.kron(H, H)
step1 = IX @ basis("00")
step2 = HH @ step1
display(Math(r"I \otimes X = " + latex_matrix(IX) + r",\quad H \otimes H = " + latex_matrix(HH)))
display(Math(r"(I \otimes X)\ket{00} = " + latex_matrix(step1.reshape(1, -1))
             + r",\quad (H \otimes H)\ket{01} = " + latex_matrix(step2.reshape(1, -1))))
display(Math(r"(H \otimes H)(I \otimes X) = " + latex_matrix(HH @ IX)))
print("Problem 8  --  (I X)|00> == |01> ?", np.allclose(step1, basis("01")))
print("Problem 8  --  (H H)|01> == |+>|-> ?", np.allclose(step2, np.kron(plus, minus)))
```

$$I \otimes X = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}, \quad H \otimes H = \begin{bmatrix} 0.5 & 0.5 & 0.5 & 0.5 \\ 0.5 & -0.5 & 0.5 & -0.5 \\ 0.5 & 0.5 & -0.5 & -0.5 \\ 0.5 & -0.5 & -0.5 & 0.5 \end{bmatrix}$$

$$(I \otimes X)\,|00\rangle = \begin{bmatrix} 0 & 1 & 0 & 0 \end{bmatrix}, \quad (H \otimes H)\,|01\rangle = \begin{bmatrix} 0.5 & -0.5 & 0.5 & -0.5 \end{bmatrix}$$

$$(H \otimes H)(I \otimes X) = \begin{bmatrix} 0.5 & 0.5 & 0.5 & 0.5 \\ -0.5 & 0.5 & -0.5 & 0.5 \\ 0.5 & 0.5 & -0.5 & -0.5 \\ -0.5 & 0.5 & 0.5 & -0.5 \end{bmatrix}$$

```
Problem 8  --  (I X)|00> == |01> ? True
Problem 8  --  (H H)|01> == |+>|-> ? True
```

1.10 9. Hadamard Gate Applied to 2-Qubit Basis State

[13]:
```
# --- Problem 9: H^{(x)2}|01> from the formula 1/2^{n/2} sum_x (-1)^{x.y} |x> ---
n, y = 2, 0b01
dot = lambda x, y: bin(x & y).count("1") % 2   # bitwise inner product mod 2
formula = np.array([(-1) ** dot(x, y) for x in range(2 ** n)]) / 2 ** (n / 2)
for x in range(2 ** n):
    print(f"  x = {x:02b}:  x.y = {dot(x, y)},  sign {(-1) ** dot(x, y):+d}")
display(Math(r"H^{\otimes 2}\ket{01} = " + latex_matrix(formula.reshape(1, -1))))
print("Problem 9  --  formula == |+>|-> ?", np.allclose(formula, np.kron(plus, minus)))
print("Problem 9  --  formula == (H H)|01> (matrix, Problem 8) ?", np.allclose(formula, HH @
  ↪basis("01")))
```

```
  x = 00:  x.y = 0,  sign +1
  x = 01:  x.y = 1,  sign -1
  x = 10:  x.y = 0,  sign +1
  x = 11:  x.y = 1,  sign -1
```

$$H^{\otimes 2}\,|01\rangle = \begin{bmatrix} 0.5 & -0.5 & 0.5 & -0.5 \end{bmatrix}$$

```
Problem 9  --  formula == |+>|-> ? True
Problem 9  --  formula == (H H)|01> (matrix, Problem 8) ? True
```

1.11 10. Non-Constant Function in the Deutsch Algorithm

[14]:
```
# --- Problem 10: the balanced f_B(x) = x in the Deutsch algorithm ---
U_B = U_f(FUNCS["B"])
after_oracle = U_B @ np.kron(plus, minus)
display(Math(r"U_{f_B}\ket{+}\ket{-} = " + latex_matrix(after_oracle.reshape(1, -1))))
print("Problem 10  --  U_f|+>|-> == |->|-> ?", np.allclose(after_oracle, np.kron(minus, minus)))
final = np.kron(H, I2) @ after_oracle
print("Problem 10  --  after H on the MSB: |1>|-> ?", np.allclose(final, np.kron(ket1, minus)))

print("---------------")


def deutsch(name):
    """Figure 4 (IBM-Q convention: q1 = x = MSB, q0 = y = LSB), oracle built from gates."""
    qc = QuantumCircuit(2, 1)
    qc.x(0)
    qc.h([0, 1])
    qc.barrier()
    if name == "B":
        qc.cx(1, 0)
    elif name == "C":
```

```
        qc.x(1); qc.cx(1, 0); qc.x(1)
    elif name == "D":
        qc.x(0)
    qc.barrier()
    qc.h(1)
    qc.measure(1, 0)
    return qc


for name in FUNCS:
    print(f"Problem 10  --  f_{name}: MSB measured = {counts(deutsch(name))}")
```

$U_{f_B} |+\rangle |-\rangle = [0.5 \quad -0.5 \quad -0.5 \quad 0.5]$

```
Problem 10  --  U_f|+>|-> == |->|-> ? True
Problem 10  --  after H on the MSB: |1>|-> ? True
---------------
Problem 10  --  f_A: MSB measured = {'0': 1024}
Problem 10  --  f_B: MSB measured = {'1': 1024}
Problem 10  --  f_C: MSB measured = {'1': 1024}
Problem 10  --  f_D: MSB measured = {'0': 1024}
```